

Chapter 8: Data Structure in python

Outline

- What is data structure?
- Lists
- Tuples
- Sets
- Dictionary

What is a Data structure?

- In computer science, a **data structure** is a particular way of **organizing data** in a computer so that it can be used efficiently
- The basic data structures in Python include **list, set, tuples,** and **dictionary**. Each of the data structures is unique in its own way.
- Data structures are “containers” that organize and group data according to type.
- List, tuple, Set, and Dictionary is the focus of this chaoter

List

What is List?

- A **List** is a kind of Collection
- A collection allows us to put many values in a single “variable”
- A collection is nice because we can carry many values around in one convenient package.
- Lists are used to store multiple items in a single variable.
- Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 being **Tuple**, **Set** and **Dictionary**, all with different qualities and usage.

Create List

- A list is a container that stores a sequence of values
- Suppose you write a program that reads a sequence of score of 10 students and prints out the sequence, spotting the largest score, like this: 50, 54 , 75.5, 99, 80, 44.5, 77, 85, 82, 66
- How do you store such sequences?
- How do you print what is stored?
- How do you spot the largest in the sequence?

```
Score = [50, 54, 75.5, 99, 80, 44.5, 77, 85, 82, 66]  
print("The sequence of scores is: ", Score)  
print ("The largest in the sequence is ", max(Score)) # output?
```

- The square brackets indicate that we are creating a list.
- The items are stored in the order they are provided. You will want to store the list in a variable so that you can access it later.
- The max() function helps you to spot the largest in the sequence

Accessing List Elements

- A list is a sequence of elements, each of which has an integer position or index.
- To access a list element, you specify which index you want to use.
 - done with the subscript operator (`[]`) in the same way that you access individual characters in a string.
 - For example, `print(Score[3])` # Prints the element at index 3
 - Note that both lists and strings are sequences, and the `[]` operator can be used to access an element in any sequence.
 - A list index must be at least zero and less than the number of elements in the list
 - **Takeaway question-**
 - What type of error is in the statement: `print(Score[10])`?
 - What built in function do you use to find the size of a list like `Score`?
- There are two differences between lists and strings.
 - **Lists** can hold values of **any type**, whereas **strings** are sequences of **characters**.
 - **Lists** are **mutable**: you can replace one list element with another, where as **Strings** are **immutable**— you cannot change the characters in the sequence.

Traversing Lists

- There are two fundamental ways of visiting all elements of a list.
 - You can loop over the index values and look up each element, or
 - you can loop over the elements themselves.
- Look at a loop that traverses all index values. Given the Score list that contains 10 elements, we will want to set a variable, say *i*, to 0, 1, 2, and so on, up to 9.

Traversing Lists cont'd

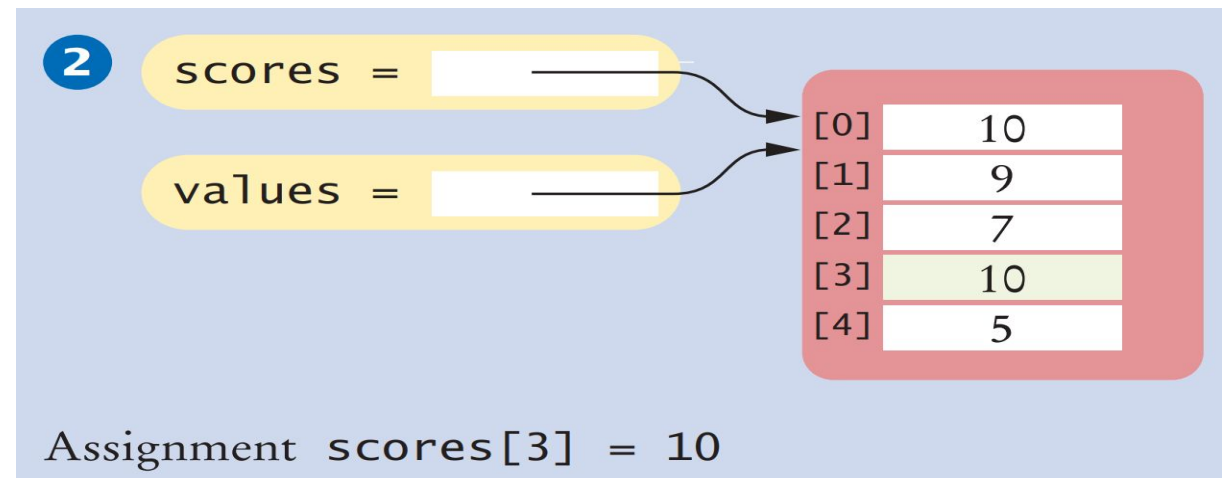
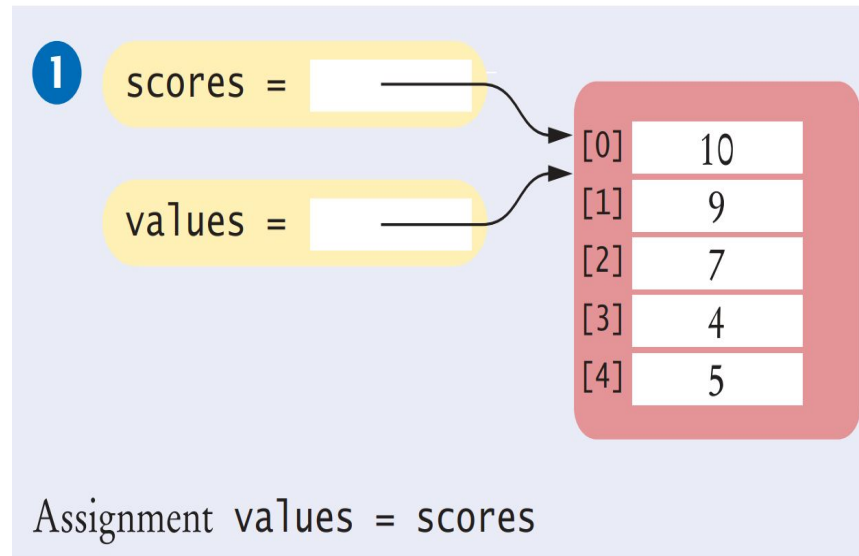
- Then the expression `Score[i]` yields each element in turn. This loop displays all index values and their corresponding elements in the `Score` list.
- `for i in range(10) :`
 - `print(i, Score[i])`
 - The variable `i` iterates through the integer values 0 through 9

Traversing Lists cont'd

- Instead of using the literal value 10 for the number of elements in the list, it is a good idea to use the len function to create a more reusable loop:
- `for i in range(len(Score)) :`
 - `print(i, Score[i])`
- If you don't need the index values, you can iterate over the individual elements using a for loop in the form:
- `for element in Score :`
 - `print(element)`

List References

- A list reference specifies the location of a list. Copying the reference yields a second reference to the same list.



List References cont'd

- When you copy a list variable into another, both variables refer to the same list (see Figure previous slide).
- The second variable is an alias for the first because both variables reference the same list.
- `scores = [10, 9, 7, 4, 5]`
- `values = scores` # Copying list reference 1
- You can modify the list through either of the variables:
`scores[3] = 10`
`print(values[3])` # Prints 10

Takeaway Exercises

1. Define a list of integers containing the first five prime numbers.
2. Assume that the list `primes` has been initialized as described in Ex 1. What does it contain after executing the following loop?
`for i in range(2) :`
`primes[4 - i] = primes[i]`
3. Assume that the list `primes` has been initialized as described in Ex 1. What does it contain after executing the following loop?
`for i in range(5) : primes[i] = primes[i] + 1`
4. Given the definition `values = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]` write statements to put the integer 10 into the elements of the list `values` with the lowest and the highest valid index.
5. Define a list containing two strings, "Yes", and "No".

Reverse Subscript

- Python, unlike many other languages, also allows you to use negative subscripts when accessing an element of a list.
- The negative subscripts provide access to the list elements in reverse order.
- For example, a subscript of `-1` provides access to the last element in the list:
 - `last = values[-1]`
 - `print("The last element in the list is", last)`
- Similarly, `values[-2]` is the second-to-last element.
- Note that, the valid range of negative subscripts is between `-1` and `-len(values)` (Figure next slide)

Reverse Subscript cont'd

values =		
	[0]	32
	[1]	54
	[2]	67
	[3]	29
	[4]	35
	[5]	60
	[6]	115
	[7]	44
	[8]	100
	[9]	65
		[-10]
		[-9]
		[-8]
		[-7]
		[-6]
		[-5]
		[-4]
		[-3]
		[-2]
		[-1]

Use Lists for Sequences of Related Items

- Lists are intended for storing sequences of values with the same meaning.
- For example, a list of test scores makes perfect sense:
 - Score = [98, 85, 100, 89, 73, 92, 83, 65, 79, 80]
- A list `personalData = ["John Q. Public", 25, 485.25, "10 wide"]` that holds a person's name, age, bank balance, and shoe size in positions 0, 1, 2, and 3 **is not impossible but it is a bad design**.
 - It would be tedious for the programmer to remember which of these data values is stored in which list location. In this situation, it is far better to use three separate variables.

List Operations

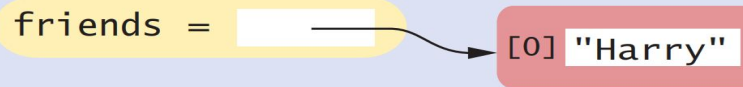
- Python has a rich set of operations that make list processing quite convenient
- **Appending Elements**
- Earlier, we created a list by specifying a sequence of initial values.
- sometimes, however, we may not know the values that will be contained in the list when it's created. In this case, we can create an empty list and add elements as needed (see next slide).
 - `friends = []` # Step 1
 - A new element can be appended to the end of the list with the append method:
`friends.append("Harry")` #Step 2
 - The size, or length, of the list increases after each call to the append method.
 - Any number of elements can be added to a list: #Step 3
 - `friends.append("Emily")`
 - `friends.append("Bob")`
 - `friends.append("Cari")`

List Operations: Appending elements

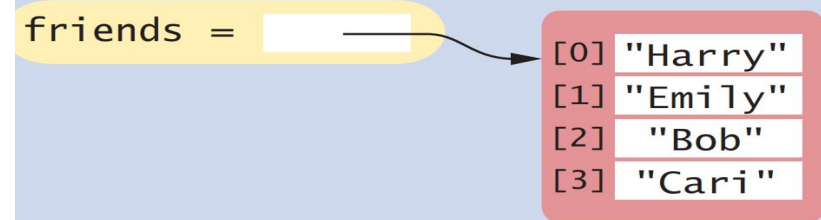
1 Create an empty list



2 Append "Harry"



3 Append additional elements



List Operations: Inserting an Element

- Use the `insert` method to insert a new element at any position in a list.
- You can add a new element to the end of a list using the list method `append`.
- If the order of the elements does not matter, appending new elements is sufficient.
- Sometimes, however, the order is important and a new element has to be inserted at a specific position in the list.
- For example, given this list, `friends = ["Harry", "Emily", "Bob", "Cari"]` # step 1 suppose we want to insert the string "Cindy" into the list following the first element, which contains the string "Harry".
- The statement `friends.insert(1, "Cindy")` # Step 2 achieves this task (Figure next slide).
- All of the elements at and following position 1 are moved down by one position to make room for the new element, which is inserted at position 1

List Operations: Inserting an Element cont'd

1 The newly created list

friends =

[0]	"Harry"
[1]	"Emily"
[2]	"Bob"
[3]	"Cari"

2 After friends.insert(1, "Cindy")

friends =

[0]	"Harry"
[1]	"Cindy"
[2]	"Emily"
[3]	"Bob"
[4]	"Cari"

New element added at index 1

Elements at indexes 1-3 moved to create slot at index 1

3 After friends.insert(5, "Bill")

friends =

[0]	"Harry"
[1]	"Cindy"
[2]	"Emily"
[3]	"Bob"
[4]	"Cari"
[5]	"Bill"

New element appended to the list

List Operations: Inserting an Element cont'd

- The index at which the new element is to be inserted must be between 0 and the number of elements currently in the list.
- For example, in a list of length 5, valid index values for the insertion are 0, 1, 2, 3, 4, and 5.
- The element is inserted before the element at the given index, except when the index is equal to the number of elements in the list.
- Then it is appended after the last element:
 - `friends.insert(5, "Bill")` #Step 3 (previous slide)
 - This is the same as if we had used the `append` method.

List Operations: Finding an Element

- If you simply want to know whether an element is present in a list, use the `in` operator:
 - if "Cindy" in friends :
 - `print("She's a friend")`
- Often, you want to know the position at which an element occurs. The `index` method yields the index of the first match.
- For example,
 - `friends = ["Harry", "Emily", "Bob", "Cari", "Emily"]`
 - `n = friends.index("Emily")` # Sets n to 1
- If a value occurs more than once, you may want to find the position of all occurrences. You can call the `index` method and specify a starting position for the search. Here, we start the search after the index of the previous match:
 - `n2 = friends.index("Emily", n + 1)` # Sets n2 to 4

List Operations: Finding an Element cont'd

- When you call the `index` method, the element to be found must be in the list **or a runtime exception occurs**.
- It is usually a good idea to test with the `in` operator before calling the `index` method:

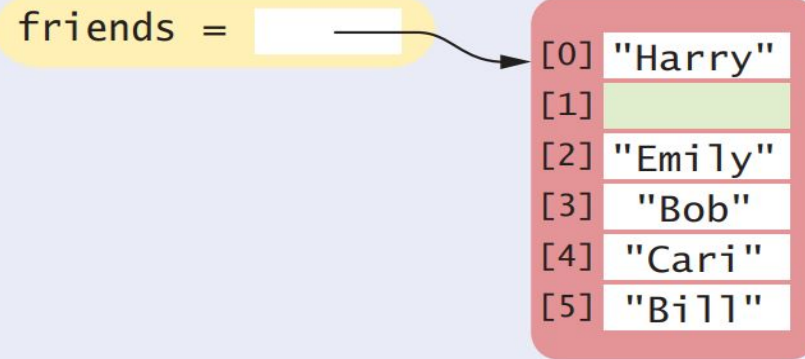
```
if "Cindy" in friends :  
    n = friends.index("Cindy")  
else : n = -1
```

List Operations: Removing an Element

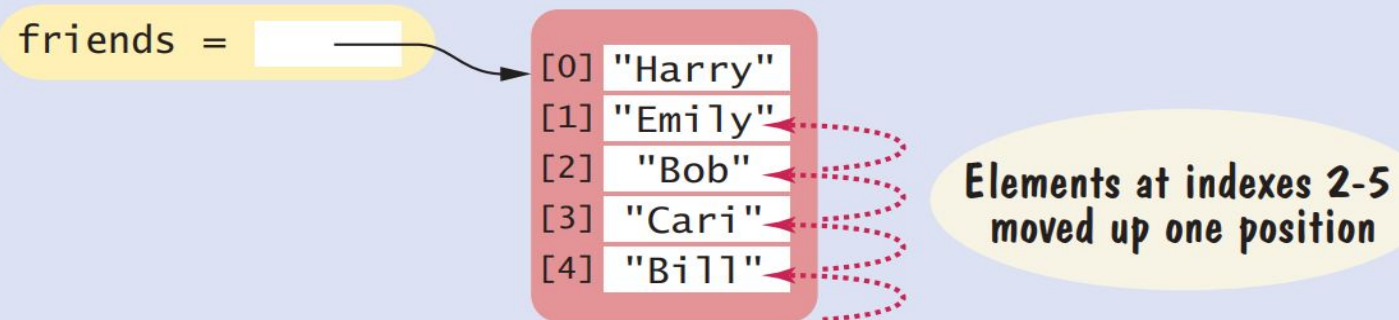
- The pop method removes the element of a list at a given position.
- For example, suppose we start with the list
 - `friends = ["Harry", "Cindy", "Emily", "Bob", "Cari", "Bill"]`
- To remove the element at index position 1 ("Cindy") in the friends list, you use the command
 - `friends.pop(1)`
- All of the elements following the removed element are moved up one position to close the gap.
- The size of the list is reduced by 1 (Next slide)

List Operations: Removing an Element cont'd

- 1 The item at index 1 is removed



- 2 The items following the removed element are moved up one position



List Operations: Removing an Element cont'd

- The index passed to the pop method must be within the valid range.
- The element removed from the list is returned by the pop method. This allows you to combine two operations in one—accessing the element and removing it:
 - `print("The removed item is", friends.pop(1))`
- If you call the pop method without an argument, it removes and returns the last element of the list.
- For example,
 - `friends.pop()` #removes "Bill".
- The remove method removes an element by value instead of by position. For example, suppose we want to remove the string "Cari" from the friends list but we do not know where it's located in the list. Instead of having to find the position, we can use the remove method:
 - `friends.remove("Cari")`

List Operations: Removing an Element cont'd

- Note that the value being removed must be in the list or an exception is raised.
- To avoid a run-time error, you should first verify that the element is in the list before attempting to remove it:
- `element = "Cari"`
- `if element in friends :`
 - `friends.remove(element)`

List Operations: Concatenation and Replication

- Concatenation of two lists produces a new list that contains the elements of the first list, followed by the elements of the second. Example:
- `myFriends = ["Fritz", "Cindy"]`
- `yourFriends = ["Lee", "Pat"]`
- Now we want to create a new list that combines the two. Two lists can be concatenated by using the plus (+) operator:
- `ourFriends = myFriends + yourFriends` # Sets ourFriends to ["Fritz", "Cindy", "Lee", "Pat"]
- If you want to concatenate the same list multiple times, use the replication operator (*). For example,
- `monthInQuarter = [1, 2, 3] * 4` # The list is [1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
- As with string replication, you can have an integer on either side of the * operator.
- The integer specifies how many copies of the list should be concatenated.

List Operations: Equality testing

- You can use the `==` operator to compare whether two lists have the same elements, in the same order.
- For example,
- `[1, 4, 9] == [1, 4, 9]` is `True`,
- but `[1, 4, 9] == [4, 1, 9]` is `False`.
- The opposite of `==` is `!=`.
- The expression `[1, 4, 9] != [4, 9]` is `True`.

List Operations: Sum, Maximum, Minimum, and Sorting

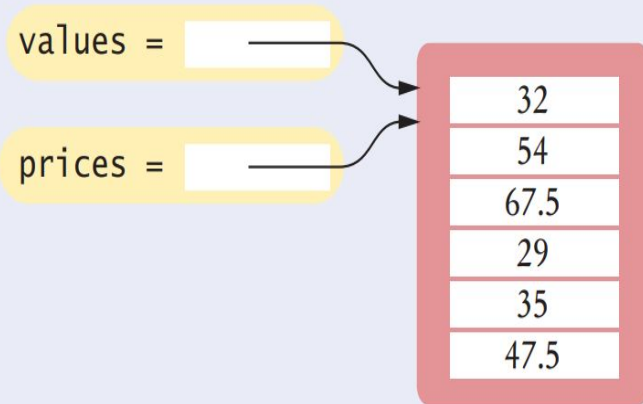
- If you have a list of numbers, the sum function yields the sum of all values in the list.
- For example:
- `sum([1, 4, 9, 16])` # Yields 30
- For a list of numbers or strings, the max and min functions return the largest and smallest value:
- `max([1, 16, 9, 4])` # Yields 16
- `min("Fred", "Ann", "Sue")` # Yields "Ann"
- The sort method sorts a list of numbers or strings. For example,
- `values = [1, 16, 9, 4]`
- `values.sort()` # Now values is [1, 4, 9, 16]

List Operations: Copying Lists

- List variables do not themselves hold list elements. They hold a reference to the actual list.
- If you copy the reference, you get another reference to the same list (see next slide):
prices = values # 1: You can modify the list through either of the two references.
- Sometimes, you want to make a copy of a list; that is, a new list that has the same elements in the same order as a given list. Use the `list` function:
prices = list(values) #2 Now, values and prices refer to different lists.
- Right after the copy, both lists have the same contents. **But you can modify either without affecting the other.**
- The list function can be used to make a list out of any sequence. For example,
characters = list("Hello") # The list is ["H", "e", "l", "l", "o"]

List Operations: Copying Lists cont'd

1 After the assignment `prices = values`



2 After calling `prices = list(values)`

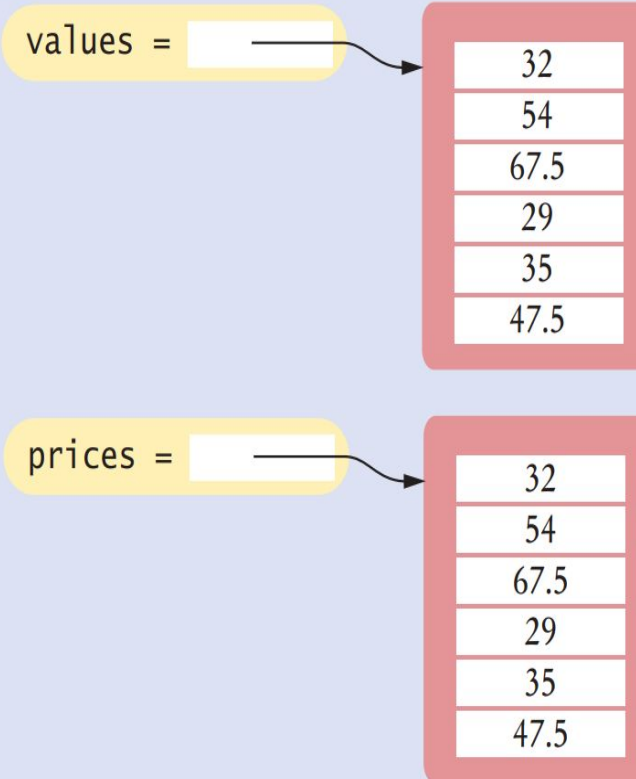


Table 1 Common Functions and Operators Used with Lists

Operation	Description
<code>[]</code> <code>[<i>elem</i>₁, <i>elem</i>₂, ..., <i>elem</i>_{<i>n</i>}]</code>	Creates a new empty list or a list that contains the initial elements provided.
<code>len(<i>l</i>)</code>	Returns the number of elements in list <i>l</i> .
<code>list(sequence)</code>	Creates a new list containing all elements of the sequence.
<code>values * num</code>	Creates a new list by replicating the elements in the values list num times.
<code>values + moreValues</code>	Creates a new list by concatenating elements in both lists.
<code><i>l</i>[from : to]</code>	Creates a sublist from a subsequence of elements in list <i>l</i> starting at position from and going through but not including the element at position to. Both from and to are optional. (See Special Topic 6.2.)
<code>sum(<i>l</i>)</code>	Computes the sum of the values in list <i>l</i> .
<code>min(<i>l</i>)</code> <code>max(<i>l</i>)</code>	Returns the minimum or maximum value in list <i>l</i> .
<code><i>l</i>₁ == <i>l</i>₂</code>	Tests whether two lists have the same elements, in the same order.

Table 2 Common List Methods

Method	Description
<code>l.pop()</code> <code>l.pop(<i>position</i>)</code>	Removes the last element from the list or from the given position. All elements following the given position are moved up one place.
<code>l.insert(<i>position</i>, <i>element</i>)</code>	Inserts the element at the given position in the list. All elements at and following the given position are moved down.
<code>l.append(<i>element</i>)</code>	Appends the element to the end of the list.
<code>l.index(<i>element</i>)</code>	Returns the position of the given element in the list. The element must be in the list.
<code>l.remove(<i>element</i>)</code>	Removes the given element from the list and moves all elements following it up one position.
<code>l.sort()</code>	Sorts the elements in the list from smallest to largest.

Takeaway Exercises

1. What is the contents of names after the operations
`names = ["Fritz"]`
`names.insert(1, "Ann")`
`names.insert(0, "Sue")`
`names.pop(2)`
`names.append("Lee")`
2. How can you remove all occurrences of an element from a list?
3. If `a` and `b` are lists such that `a + b == b + a`, is it necessarily true that `a == b`?
4. What is the difference between `[2] * 3` and `2 * [3]`?
5. How can you make a copy of a list without the list function?
6. How can you compute the average of a non-empty list of numbers?

List Operations: slice

- Sometimes you want to look at a part of a list. For example, suppose you are given a list of temperatures, one per month:
- `temperatures = [18, 21, 24, 28, 33, 39, 40, 39, 36, 30, 22, 18]` You are interested in the temperatures only for the third quarter, with index values 6, 7, and 8.
- You can use Python's slice operator (`:`) to obtain them.
- `thirdQuarter = temperatures[6 : 9]` The arguments for the slice operator are the first index to include in the slice, followed by the first index to exclude.
- This may seem a curious arrangement, but it has a useful property. The length of the slice `temperatures[a : b]` is the difference $b - a$. In our case, the difference $9 - 6 = 3$ is the number of months in a quarter.

List Operations: slice cont'd

- Both index values used with the slice operator (6 and 9 here) are optional. If the first index is omitted, all elements from the first element on are included.
- The slice `temperatures[:6]` contains all elements up to (but not including) position 6. That's the first half of the year.
- The slice `temperatures[6:]` includes all elements from index 6 to the end of the list; that is, the second half of the year.
- If you omit both index values, `temperatures[:]`, you get a copy of the list. You can even assign values to a slice. For example, the assignment `temperatures[6:9] = [45, 44, 40]` replaces the values for the third quarter.
- Slices work with all sequences, not just lists. They are particularly useful with strings. A slice of a string is a substring:
- `greeting = "Hello, World!"`
- `greeted = greeting[7:12] # The substring "World"`

Tuples

What is a Tuple?

- A tuple is a sequence of values much like a list.
- The values stored in a tuple **can be any type**, and they are **indexed by integers**.
- The important difference from a list is that tuples are **immutable**. Tuples are also comparable and hashable so we can sort lists of them and use tuples as key values in Python dictionaries.
- Syntactically, a tuple is a comma-separated list of values:

```
>>> t = 'a', 'b', 'c', 'd', 'e'
```

- Although it is not necessary, it is common to enclose tuples in parentheses to help us quickly identify tuples when we look at Python code:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
```

- To create a tuple with a single element, you have to include the final comma:

```
>>> t1 = ('a',)
```

```
>>> type(t1) # <type 'tuple'>
```

- Without the comma Python treats ('a') as an expression with a string in parentheses that evaluates to a string:

```
>>> t2 = ('a')
```

```
>>> type(t2) # <type 'str'>
```

Tuple cont'd

- Many of the operations defined for a list can also be used with a tuple:
 - Access an individual element of a tuple by position (element = triple[1]).
 - Obtain the number of elements in the tuple with the len function.
 - Iterate over the elements of a tuple using for loops.
 - Test for members using the `in` and `not in` operators.
- In fact, any list operation that does not modify the contents of the list can be used with a tuple.
- A tuple is simply an immutable version of a list.

Tuple cont'd

- Another way to construct a tuple is the built-in function `tuple`. With no argument, it creates an empty tuple:

```
>>> t = tuple()
```

```
>>> print(t) ()
```

- If the argument is a sequence (string, list, or tuple), the result of the call to `tuple` is a tuple with the elements of the sequence:

```
>>> t = tuple('lupins')
```

```
>>> print(t) # ('l', 'u', 'p', 'i', 'n', 's')
```

- Because `tuple` is the name of a constructor, you should avoid using it as a variable name.

Tuple cont'd

- Most list operators also work on tuples.
- The bracket operator indexes an element:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
```

```
>>> print(t[0]) # 'a'
```

- And the slice operator selects a range of elements.

```
>>> print(t[1:3]) # ('b', 'c')
```

- But if you **try to modify** one of the elements of the tuple, you get an **error**:

```
>>> t[0] = 'A' #TypeError: object doesn't support item assignment
```

- You can't modify the elements of a tuple, but you can replace one tuple with another:

```
>>> t = ('A',) + t[1:]
```

```
>>> print(t) #('A', 'b', 'c', 'd', 'e')
```

Tuple Assignment

- One of the **unique** syntactic features of the Python language is the ability to have a tuple on the left side of an assignment statement.
 - This allows you to assign more than one variable at a time when the left side is a sequence.
- Example we have a two-element list (which is a sequence) and let's assign the first and second elements of the sequence to the variables `x` and `y` in a single statement.

```
>>> m = [ 'have', 'fun' ]
>>> x, y = m
>>> x # 'have'
>>> y #'fun'
>>>
```

Python **roughly translates** the tuple assignment syntax to be the following:

```
>>> m = [ 'have', 'fun' ]
>>> x = m[0
] >>> y = m[1]
>>> x #'have'
>>> y #'fun'
>>>
```

Tuple Assignment cont'd

- Stylistically when we use a tuple on the left side of the assignment statement, we omit the parentheses, but the following is an equally valid syntax:

```
>>> m = [ 'have', 'fun' ]
```

```
>>> (x, y) = m
```

```
>>> x #'have'
```

```
>>> y #'fun'
```

```
>>>
```

Tuple Assignment cont'd

- A particularly clever application of tuple assignment allows us to **swap** the values of two variables in a **single** statement:

```
>>> a, b = b, a
```

- Both sides of this statement are tuples, but the left side is a tuple of variables; the right side is a tuple of expressions.
- Each value on the right side is assigned to its respective variable on the left side.
- All the expressions on the right side are evaluated before any of the assignments.
- The number of variables on the left and the number of values on the right must be the **same**:

```
>>> a, b = 1, 2, 3 #ValueError: too many values to unpack
```

Tuple Assignment cont'd

- More generally, the right side can be any kind of sequence (string, list, or tuple).
- For example, to split an email address into a user name and a domain, you could write:

```
>>> addr = 'monty@python.org'
```

- ```
>>> uname, domain = addr.split('@')
```
- The return value from `split` is a list with two elements; the first element is assigned to `uname`, the second to `domain`.

```
>>> print(uname) #monty
```

```
>>> print(domain) #python.org
```

# Set

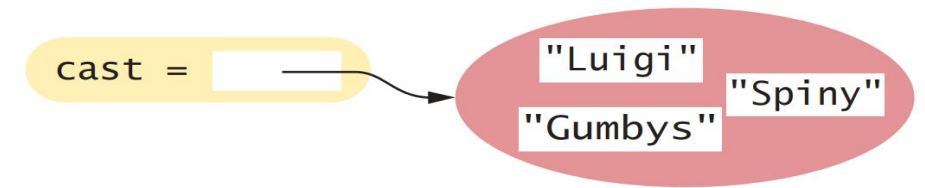
# What is Set?

- A set is a container that stores a collection of **unique values**.
- Unlike a list, the elements or members of the set are **not stored in any particular order and cannot be accessed by position**.
- The operations available for use with a set, same as the operations performed on sets in mathematics.
- Because sets need not maintain a particular order, set operations are **much faster** than the equivalent list operations

# Creating and Using Sets

- To create a set with initial elements, you can specify the elements enclosed in braces, just like in mathematics:

```
cast = { "Luigi", "Gumbys", "Spiny" }
```



- Alternatively, you can use the `set` function to convert any sequence into a set:

```
names = ["Luigi", "Gumbys", "Spiny"]
cast = set(names)
```

- For historical reasons, you cannot use `{}` to make an empty set in Python. Instead, use the `set` function with no arguments:

```
cast1 = set()
```

# Creating and Using Sets cont'd

- As with any container, you can use the `len` function to obtain the number of elements `in a set`:  
`numberOfCharacters = len(cast)`
- To determine whether an element is contained in the set, use the `in operator` or its inverse, the `not in operator`:  
`if "Luigi" in cast :`  
`print("Luigi is a character in Monty Python's Flying Circus.")`  
`else :`  
`print("Luigi is not a character in the show.")`
- Because sets are unordered, you cannot access the elements of a set by position
- Instead, use a for loop to iterate over the individual elements:  
`print("The cast of characters includes:")`  
`for character in cast :`  
`print(character)`

# Creating and Using Sets cont'd

- The order in which the elements of the set are visited depends on how they are stored internally.
- For example, the loop (previous slide) displays the following:
- The cast of characters includes: Spiny Gumbys Luigi
- Note that the order of the elements in the output is different from the order in which the set was created.
- When you try it you may get a different sequence (obviously order is not an issue in a set, but uniqueness)

# Creating and Using Sets cont'd

- The fact that sets do not retain the initial ordering is not a problem when working with sets.
- In fact, the lack of an ordering makes it possible to implement set operations very efficiently.
- However, you usually want to display the elements in sorted order.
- Use the `sorted` function, which returns a `list` (not a `set`) of the elements in sorted order. The following loop prints the cast in sorted order:

```
for character in sorted(cast) :
 print(character)
```

```
myList = []
for character in sorted(cast):
 myList.append(character)
print(myList)
myList2 = sorted(cast)
print(myList2)
```

# Adding and Removing Elements

- Like lists, sets are **mutable collections**, so you can add and remove elements.
- For example, suppose we need to add more characters to the set cast created in the previous slide. Use the **add** method to add elements:

```
cast = set(["Luigi", "Gumbys", "Spiny"]) #step1
```

```
cast.add("Arthur") # step2
```

```
cast.add("Spiny") #step 3
```

```
cast.discard("Arthur") #step 4
```

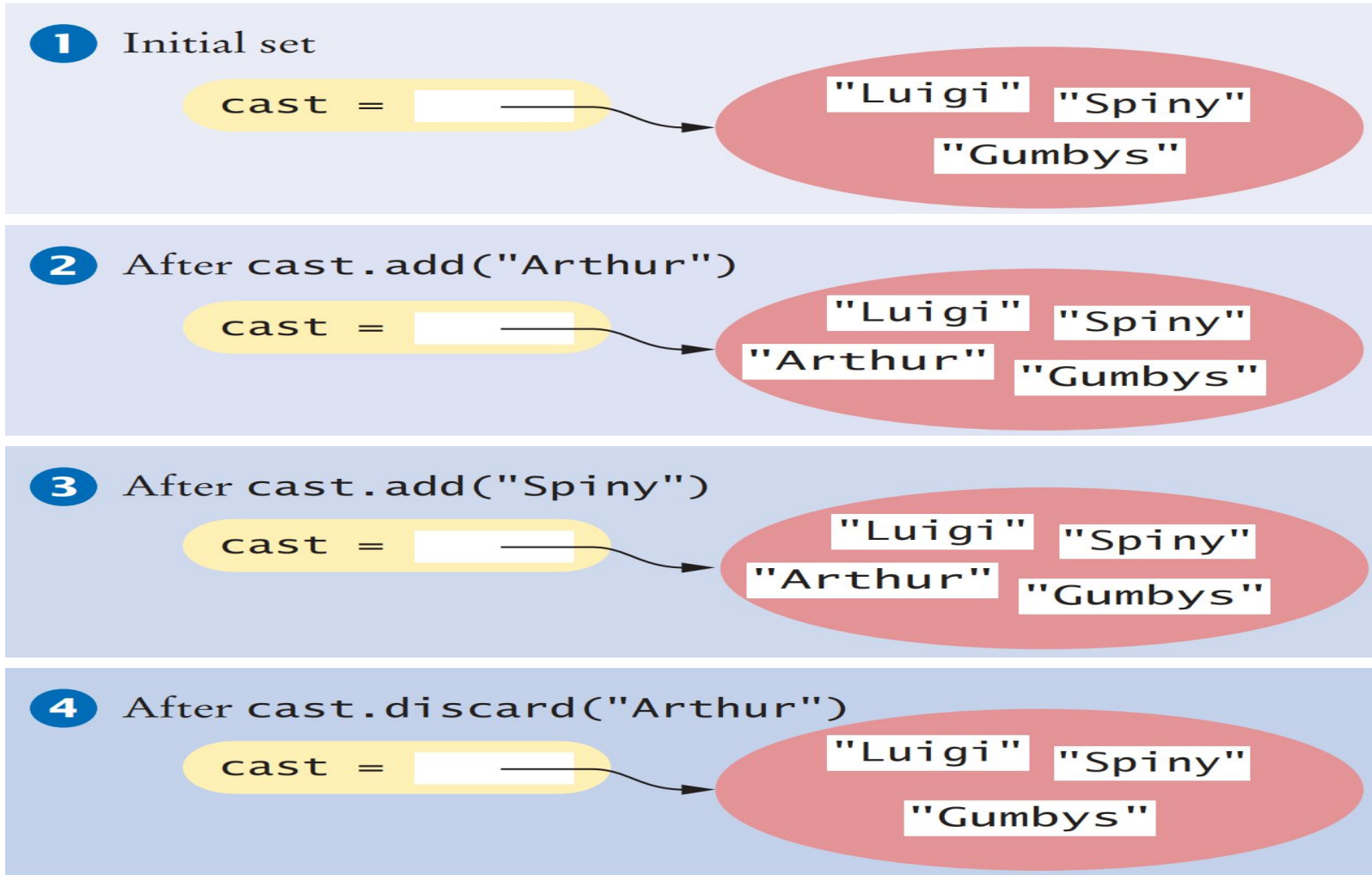
- **discard** has no effect if the given element is not a member of the set:  

```
cast.discard("The Colonel") # Has no effect
```
- The **remove** method, on the other hand, removes an element **if it exists**, but raises an exception if the given element is not a member of the set: 

```
cast.remove("The Colonel") # Raises an exception
```
- Finally, the **clear** method removes all elements of a set, leaving the empty set:  

```
cast.clear() # cast now has size 0
```

# Adding and Removing Elements cont'd



# Set operations

| Table 1 Common Set Operations                                                                                              |                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Operation                                                                                                                  | Description                                                                                                                                                      |
| <code>s = set()</code><br><code>s = set(seq)</code><br><code>s = {e<sub>1</sub>, e<sub>2</sub>, ..., e<sub>n</sub>}</code> | Creates a new set that is either empty, a duplicate copy of sequence <i>seq</i> , or that contains the initial elements provided.                                |
| <code>len(s)</code>                                                                                                        | Returns the number of elements in set <i>s</i> .                                                                                                                 |
| <code>element in s</code><br><code>element not in s</code>                                                                 | Determines whether <i>element</i> is in the set.                                                                                                                 |
| <code>s.add(element)</code>                                                                                                | Adds a new element to the set. If the element is already in the set, no action is taken.                                                                         |
| <code>s.discard(element)</code><br><code>s.remove(element)</code>                                                          | Removes an element from the set. If the element is not a member of the set, <code>discard</code> has no effect, but <code>remove</code> will raise an exception. |
| <code>s.clear()</code>                                                                                                     | Removes all elements from a set.                                                                                                                                 |

# Set operations cont'd

|                                            |                                                                                                  |
|--------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>s.issubset(t)</code>                 | Returns a Boolean indicating whether set <i>s</i> is a subset of set <i>t</i> .                  |
| <code>s == t</code><br><code>s != t</code> | Returns a Boolean indicating whether set <i>s</i> is equal or not equal to set <i>t</i> .        |
| <code>s.union(t)</code>                    | Returns a new set that contains all elements in set <i>s</i> and set <i>t</i> .                  |
| <code>s.intersection(t)</code>             | Returns a new set that contains elements that are in <i>both</i> set <i>s</i> and set <i>t</i> . |
| <code>s.difference(t)</code>               | Returns a new set that contains elements in <i>s</i> that are not in set <i>t</i> .              |

# Takeaway Exercises

1. What is the output of the following code snippet?

```
s = set()
for i in range(1, 5) :
 s.add(i * i)
 s.add(-i)
print(s)
```

2. What is the output of the following code snippet?

```
s = set()
for i in range(0, 20, 2) :
 s.add(i)
for i in range(0, 20, 3) :
 s.discard(i)
print(s)
```

# Dictionaries

# What is a dictionary?

- A dictionary is a container that keeps associations between **keys** and **values**.
- Every key in the dictionary has an associated value.
- Keys are unique, but a value may be associated with several keys.
- The dictionary structure is also known as a map because it maps a unique key to a value.
- It stores the keys, values, and the associations between them

# Set and Dictionary Literals

Set and dictionary elements are enclosed in braces.

A set

```
colors = { "Red", "Green", "Blue" }
```

Key

Value

```
favoriteColors = { "Romeo": "Green", "Adam": "Red" }
```

Dictionaries contain key/value pairs.

```
emptyDict = {}
```

An empty pair of braces is a dictionary.

# Creating Dictionaries

- Suppose you need to write a program that looks up the phone number for a person in your mobile phone's contact list.
- You can use a dictionary where the names are keys and the phone numbers are values, where a person can have more than one number
- The dictionary also allows you to associate more than one person with a given number
- we create a small dictionary for a contact list that contains four items
- `contacts = { "Fred": 7235591, "Mary": 3841212, "Bob": 3841212, "Sarah": 2213278 }`
- Each key/value pair is separated by a colon. You enclose the key/value pairs in braces, just as you would when forming a set.
- When the braces contain key/value pairs, they denote a dictionary, not a set. The only ambiguous case is an empty `{}`. By convention, it denotes an empty dictionary, **not an empty set**

# Accessing Dictionary Values

- The subscript operator `[]` is used to return the value associated with a **key**.
- The statement `print("Fred's number is", contacts["Fred"])` prints 7235591.
- Note that the dictionary is not a sequence-type container like a list.
- Even though the subscript operator is used with a dictionary, you cannot access the items by **index or position**.
- A value can only be accessed using its **associated key**.
- The key supplied to the subscript operator **must be a valid key** in the dictionary **or a `KeyError` exception will be raised**.
- To find out whether a key is present in the dictionary, use the **`in`** (or **`not in`**) operator:

if "John" in contacts :

```
 print("John's number is", contacts["John"])
```

else :

```
 print("John is not in my contact list.")
```

# Adding and Modifying Items

- A dictionary is a mutable container. That is, you can change its contents after it has been created.
- You can add a new item using the subscript operator [] much as you would with a list

```
contacts["John"] = 4578102# case 1
```

- To change the value associated with a given key, set a new value using the [] operator on an existing key:

```
contacts["John"] = 2228102 #case 2
```

- You can create an empty dictionary like this:

```
favoriteColors = {}
```

- and add new items as needed:

```
favoriteColors["Juliet"] = "Blue"
```

```
favoriteColors["Adam"] = "Red"
```

```
favoriteColors["Eve"] = "Blue"
```

```
favoriteColors["Romeo"] = "Green"
```

# Removing Items

- To remove an item from a dictionary, call the `pop` method with the `key` as the `argument`:

```
contacts.pop("Fred") # This removes the entire item, both the key and its associated value
```

- The `pop` method returns the value of the item being removed, so you can use it or store it in a variable:

```
fredsNumber = contacts.pop("Fred")
```

- If the key is `not` in the dictionary, the `pop` method **raises a `KeyError` exception**.
- To prevent the exception from being raised, you can test for the key in the dictionary:

```
if "Fred" in contacts :
 contacts.pop("Fred")
```

# Traversing a Dictionary

- You can iterate over the individual keys in a dictionary using a for loop:

```
print("My Contacts:")
```

```
for key in contacts :
```

```
 print(key)
```

- The result of this code fragment is shown below:

My Contacts:

Sarah

Bob

John

Mary

Fred

- Note that the dictionary stores its items in an order that is optimized for efficiency, which may not be the order in which they were added

# Takeaway Exercises

1. Create and initialize a dictionary that maps the English words “one” through “five” to the numbers 1 through 5.
2. Write a loop that prints all values of the dictionary that you defined in Ex 1
3. What is the difference between a list and a dictionary?
4. Suppose you want to track how many times each word occurs in a document. Describe how a dictionary can be used for this task

## Table 2 Common Dictionary Operations

| Operation                                                   | Returns                                                                                                                                                                |
|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $d = \text{dict}()$<br>$d = \text{dict}(c)$                 | Creates a new empty dictionary or a duplicate copy of dictionary $c$ .                                                                                                 |
| $d = \{\}$<br>$d = \{k_1: v_1, k_2: v_2, \dots, k_n: v_n\}$ | Creates a new empty dictionary or a dictionary that contains the initial items provided. Each item consists of a key ( $k$ ) and a value ( $v$ ) separated by a colon. |
| $\text{len}(d)$                                             | Returns the number of items in dictionary $d$ .                                                                                                                        |
| $key \text{ in } d$<br>$key \text{ not in } d$              | Determines if the key is in the dictionary.                                                                                                                            |
| $d[key] = value$                                            | Adds a new <i>key/value</i> item to the dictionary if the <i>key</i> does not exist. If the key does exist, it modifies the value associated with the key.             |
| $x = d[key]$                                                | Returns the value associated with the given key. The key must exist or an exception is raised.                                                                         |

# Common dictionary operations cont'd

|                                  |                                                                                                                                            |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>d.get(key, default)</code> | Returns the value associated with the given key, or the default value if the key is not present.                                           |
| <code>d.pop(key)</code>          | Removes the key and its associated value from the dictionary that contains the given key or raises an exception if the key is not present. |
| <code>d.values()</code>          | Returns a sequence containing all values of the dictionary.                                                                                |

# Takeaway Exercises

1. Write a function that takes two string arguments and returns
  1. a set consisting of the upper- and lowercase letters that are contained in both strings.
  2. a set consisting of the upper- and lowercase letters that are not contained in either string.
  3. a set consisting of all non-letter characters contained in both strings.
2. Given a dictionary  
`gradeCounts = { "A": 8, "D": 3, "B": 15, "F": 2, "C": 6 }`  
write the Python statement(s) to print:
  1. all the keys.
  2. all the values.
  3. all the key and value pairs.
  4. all of the key and value pairs in key order.
  5. the average value

# End of Chapter 8